RETAILMART V3 · SQL

Data Types, Constraints & Keys

— Siraj Sir

📱 WhatsApp Announcement

DATA TYPES + CONSTRAINTS & KEYS: Make Your Data Correct and Safe!

Yesterday every column was TEXT - great for learning structure, dangerous in real life. Today you give each column its PROPER type (and ALTER yesterday's TEXT columns into them), then add RULES that make bad data impossible.

Today's Focus:

- Data Types - INT, NUMERIC, DATE, TIMESTAMP, BOOLEAN, and the type decision framework
- ALTER TABLE ... TYPE ... USING - convert yesterday's TEXT columns to real types
- NOT NULL, DEFAULT, UNIQUE, CHECK - data integrity constraints
- PRIMARY KEY - identify every row. FOREIGN KEY - link tables. ON DELETE actions.

Important: ALL practice today is in YOUR practice database (accio_NN). You never change RetailMart's tables - you study its type choices and build your own.

Course Portal: https://starlordali4444.github.io/postgresql/

Today your tables go from 'a pile of text' to a real, self-defending database. Let's go!
- Siraj Sir
#Day4 #DataTypes #Constraints #Keys #PostgreSQL

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap - Day 3 (DDL + DML)	5 mins	You built TEXT-only tables, You INSERT/UPDATE/DELETE/TRUNCATE, Why all-TEXT hurt
Topic 1: Data Types	40 mins	Numeric, Text, Date/Time, Boolean, Special, The Type Decision Framework, ALTER TEXT -> proper types with USING
Practice Block 1 - Data Types	25 mins	4 problems: pick types, fix wrong types, ALTER TEXT columns
Topic 2: Constraints & Keys	45 mins	NOT NULL, DEFAULT, UNIQUE, CHECK, PRIMARY KEY, FOREIGN KEY, ON DELETE RESTRICT / CASCADE / SET NULL
Practice Block 2 - Constraints & Keys	25 mins	4 problems: enforce rules, link tables, demo violations
Wrap-up	10 mins	Summary, self-check, homework, Filtering & Sorting preview

Where We Left Off

```
Yesterday you:
  - Built tables with CREATE SCHEMA / CREATE TABLE (DDL)
  - Gave EVERY column the type TEXT (focus on structure, not types)
  - Filled tables with INSERT, changed rows with UPDATE,
    removed rows with DELETE, emptied tables with TRUNCATE (DML)

The pain you felt with all-TEXT columns:
  - Math was clumsy: amount * 1.1 fails (amount is TEXT)
  - Comparisons sorted like words: WHERE total > '900' MISSED '1000'
    ('1000' < '900' character-by-character)

Today we fix BOTH problems:
  Topic 1 (Data Types): ALTER those TEXT columns to INT/NUMERIC/DATE/BOOLEAN.
  Topic 2 (Constraints & Keys): add RULES so bad data can never get in.
```

STORY

The Rs. 34 Lakh Bug - Why TEXT-For-Everything Is Dangerous (1/2)

A fintech startup in Bengaluru stored transaction amounts as TEXT (exactly like yesterday's tables). Their reasoning: 'We will convert to numbers when we need to calculate.' Sound familiar?

```
[How TEXT Destroyed Their Finances]
-- Their payment INSERTs looked fine (all TEXT):
INSERT INTO payments (user_id, amount) VALUES ('1001', '15000');


-- But one day a frontend bug sent these - and TEXT ACCEPTED them all:
INSERT INTO payments (user_id, amount) VALUES ('1003', 'Rs.15,000');
INSERT INTO payments (user_id, amount) VALUES ('1004', 'fifteen thousand');
```

STORY

The Rs. 34 Lakh Bug - Why TEXT-For-Everything Is Dangerous (2/2)

```
-- Quarter-end report:
SELECT SUM(amount::NUMERIC) FROM payments;
-- ERROR: invalid input syntax for type numeric: 'Rs.15,000'


-- The fix was one line - the RIGHT TYPE:
ALTER TABLE payments ALTER COLUMN amount TYPE NUMERIC(12,2)
    USING amount::numeric;
-- Now 'Rs.15,000' is REJECTED at INSERT time, not discovered 8 months later.
```

Data types are your FIRST line of defense. The right type REJECTS bad data at the gate so you never clean it later. That is why we do NOT leave columns as TEXT in real systems - we choose the correct type for each.

Whole Numbers and Auto-IDs

```
Type        | Range / Behavior                | Use For
------------+---------------------------------+-------------------------
SMALLINT    | -32,768 to 32,767               | age, rating, floor number
INT/INTEGER | +/- 2.1 billion                 | most IDs, quantities, counts
BIGINT      | +/- 9.2 quintillion             | views, likes, huge counters
SERIAL      | auto 1,2,3... (an auto INT)     | auto-generated primary keys
BIGSERIAL   | auto, BIGINT-sized              | huge tables

-- SERIAL auto-generates ids - you never type them:
-- INSERT INTO t (name) VALUES ('Rahul');  -- id becomes 1
-- Deleted ids are never reused (id 2 gone forever). That is intentional.

Rule: use the SMALLEST integer type that comfortably fits your data.
```

Decimals and Money

```
Type          | Precision           | Use When
--------------+---------------------+--------------------------------
NUMERIC(p,s)  | EXACT               | money, prices, tax - ALWAYS
REAL / DOUBLE | approximate         | scientific measurements only

NUMERIC(p, s): p = total digits, s = digits after the decimal point.
  NUMERIC(10,2) -> up to 99999999.99   NUMERIC(9,6) -> GPS lat/long

-- NEVER use FLOAT/REAL for money:
SELECT 0.1::REAL + 0.2::REAL;     -- 0.30000001  (WRONG)
SELECT 0.1::NUMERIC + 0.2::NUMERIC; -- 0.3       (CORRECT)
-- Over millions of transactions, float drift costs real money.
```

The Three Text Types

```
Type         | Length     | Best For
-------------+------------+-----------------------------------------
VARCHAR(n)   | up to n    | names, emails, cities, codes (most common)
TEXT         | unlimited  | reviews, descriptions, long comments
CHAR(n)      | fixed n    | almost never (pads with spaces - wasteful)

VARCHAR(100) and VARCHAR(50) use the SAME storage for a 30-char value -
the limit only REJECTS values that exceed it. Set it generously.

The phone / pincode / aadhaar trap - these are NOT numbers:
  phone '+91-098765-43210' -> has +, dashes, leading 0 -> VARCHAR
  pincode '022987' -> INT would drop the leading 0 -> VARCHAR
  Test: do you ever SUM or AVG it? No -> it is text, not a number.
```

Dates, Times, and Flags

Type	Stores	Use When
DATE	calendar date (2025-05-16)	birthday, order_date
TIMESTAMP	date + time	created_at, login, events
TIMESTAMPTZ	date + time + timezone	global apps
TIME	clock time only	shifts, schedules
INTERVAL	a duration (3 days 4 hours)	SLAs, time spans
BOOLEAN	true / false / NULL	is_active, has_ac flags

PostgreSQL expects dates as YYYY-MM-DD: '2025-05-16' (not 16/05/2025).
DATE vs TIMESTAMP: order_date = which DAY -> DATE.
page_view_at = exact moment -> TIMESTAMP.

Special (deep dive later): UUID (unguessable ids), JSONB (flexible nested),
ARRAY (many values in one cell). Covered on the JSON day.

Master these 8 and you handle 95% of columns:
SERIAL, INT, NUMERIC, VARCHAR, TEXT, DATE, TIMESTAMP, BOOLEAN.

The 10-Second Type Decision Tree

```
1. Auto-incrementing id?            -> SERIAL (or BIGSERIAL)
2. Whole number you do math on?     -> SMALLINT / INT / BIGINT
3. Money or precise decimal?        -> NUMERIC(p,s)  (never FLOAT)
4. Short text (name, email, code)?  -> VARCHAR(n)
5. Long text (review, description)? -> TEXT
6. Calendar date?                   -> DATE
7. Exact moment?                    -> TIMESTAMP
8. Yes/No?                          -> BOOLEAN
9. Looks like a number but you      -> VARCHAR(n)
   NEVER do math on it (phone/pin)?

Still unsure? Default to VARCHAR, then ALTER later - which is exactly
what we do next with yesterday's TEXT columns.
```

DEFINITION

ALTER TABLE - Turning Yesterday's TEXT Into Proper Types (1/2)

Yesterday every column was TEXT. Today you convert them to their proper types WITHOUT dropping the table - using ALTER TABLE … ALTER COLUMN … TYPE … USING. The USING clause tells PostgreSQL how to cast the existing text values.

```
[The Conversion Toolkit]
-- Convert a TEXT column to INT (cast existing values):
ALTER TABLE college.students
  ALTER COLUMN student_id TYPE INT USING student_id::int;
```

DEFINITION

ALTER TABLE - Turning Yesterday's TEXT Into Proper Types (2/2)

```
-- TEXT -> DATE:
ALTER TABLE college.students
  ALTER COLUMN date_of_birth TYPE DATE USING date_of_birth::date;


-- TEXT -> BOOLEAN:
ALTER TABLE college.students
  ALTER COLUMN is_active TYPE BOOLEAN USING is_active::boolean;


-- TEXT -> NUMERIC (money):
ALTER TABLE college.fee_payments
  ALTER COLUMN amount TYPE NUMERIC(10,2) USING amount::numeric;


-- Other ALTERs you will use:
ALTER TABLE t ADD COLUMN created_at TIMESTAMP DEFAULT now();
ALTER TABLE t RENAME COLUMN old_name TO new_name;
ALTER TABLE t DROP COLUMN unused;
```

REAL WORLD

RETAILMART TYPE AUDIT - Learn From Production

How RetailMart V3 chose types (run \d sales.orders to see for yourself):

```
[sales.orders - Type Choices Explained]
Column          | Type          | Why
----------------+---------------+---------------------------------
order_id        | SERIAL        | auto-increment primary key
cust_id         | INT           | references customers (a FK)
order_date      | DATE          | the DAY matters, not the hour
order_status    | VARCHAR(30)   | 'Delivered', 'Processing'...
gross_total     | NUMERIC(12,2) | money - MUST be exact
net_total       | NUMERIC(12,2) | gross_total - discount
payment_mode_id | INT           | references finance.payment_modes (FK)


Patterns: ids are SERIAL/INT, money is NUMERIC, day-only is DATE,
status is VARCHAR. Every choice is deliberate - and yours should be too.
```

PRACTICE BLOCK 1 - Data Types

All practice in accio_NN. Choosing the RIGHT type is the skill - every wrong type is a future bug.

EASY

SCENARIO: Yesterday you built college.students with EVERY column as TEXT. Today, give each column its proper type - without dropping the table.

YOUR TASK:

a) : Make sure college.students has a few rows (from yesterday). If empty, INSERT 2-3 rows (text values like '1', '2004-03-15', 'true').
b) : ALTER student_id from TEXT to INT (use USING student_id::int).
c) : ALTER date_of_birth from TEXT to DATE.
d) : ALTER is_active from TEXT to BOOLEAN.
e) : Run \d college.students and confirm the new types. Now student_id math and date comparisons work.

Hint: TYPE ... USING column::newtype tells PostgreSQL how to convert the existing text. If a value cannot convert, the ALTER fails - that is the type defending your data.

Answer

✅ ANSWER

```
-- (a) ensure some data exists
INSERT INTO college.students
  (student_id, first_name, last_name, email, date_of_birth, is_active)
VALUES ('1', 'Aarav', 'Sharma', 'aarav@college.edu', '2004-03-15', 'true');

-- (b)(c)(d) convert TEXT -> proper types
ALTER TABLE college.students
  ALTER COLUMN student_id TYPE INT USING student_id::int;
ALTER TABLE college.students
  ALTER COLUMN date_of_birth TYPE DATE USING date_of_birth::date;
ALTER TABLE college.students
  ALTER COLUMN is_active TYPE BOOLEAN USING is_active::boolean;

-- (e) verify
SELECT column_name, data_type FROM information_schema.columns
WHERE table_schema='college' AND table_name='students'
ORDER BY ordinal_position;
```

MEDIUM

SCENARIO: A junior developer sent this CREATE TABLE. It is full of WRONG type choices. Find and fix every mistake.

YOUR TASK: Write the corrected CREATE TABLE, execute it in accio_NN, and insert 2 rows to prove the types work.

Hint: money -> NUMERIC(12,2); phone/pincode -> VARCHAR; yes/no -> BOOLEAN; rating 1-5 -> NUMERIC(2,1); long notes -> TEXT; id -> SERIAL.

Find the 9 Wrong Types

✅ ANSWER

```
CREATE TABLE public.customer_orders (
    order_id       VARCHAR(10),    -- ids are integers
    customer_name  CHAR(200),      -- CHAR pads; names vary
    order_date     VARCHAR(20),    -- use a real date
    total_amount   INT,            -- money has paisa!
    phone          BIGINT,         -- loses +91 / leading 0
    pincode        INT,            -- loses leading 0
    is_delivered   VARCHAR(5),     -- true/false
    rating         NUMERIC(10,5),  -- rating is 1-5, not 99999.99999
    notes          VARCHAR(50)     -- notes can be long
);
```

Answer

✅ ANSWER

```
CREATE TABLE public.customer_orders (
  order_id      SERIAL PRIMARY KEY,  -- was VARCHAR(10)
  customer_name VARCHAR(100),        -- was CHAR(200)
  order_date    DATE,                -- was VARCHAR(20)
  total_amount  NUMERIC(12,2),       -- was INT (paisa lost)
  phone         VARCHAR(15),         -- was BIGINT
  pincode       VARCHAR(6),          -- was INT (leading 0)
  is_delivered  BOOLEAN,             -- was VARCHAR(5)
  rating        NUMERIC(2,1),        -- was NUMERIC(10,5)
  notes         TEXT                 -- was VARCHAR(50)
);

INSERT INTO public.customer_orders
  (customer_name, order_date, total_amount, phone, pincode,
   is_delivered, rating, notes)
VALUES
  ('Priya', '2026-01-05', 1299.50, '+91-9876543210', '400058',
   true, 4.5, 'Leave at the gate.'),
  ('Rahul', '2026-01-06', 2499.00, '9123456780', '110001',
   false, 3.5, 'Call on arrival.');
```

HARD

SCENARIO: You are designing tables for Zomato from scratch. Millions of orders per day - every wrong type haunts you at scale. Choose types deliberately.

YOUR TASK: Create these 2 tables in accio_NN with correct types and justify each:

a) : restaurants - id, name, city, latitude, longitude (28.6139 needs decimals), avg_rating (4.3 possible), is_pure_veg, opening_time, created_at.
b) : menu_items - id, restaurant_id, item_name, description (can be 500 words), price, is_veg, prep_time_minutes, calories.

Hint: lat/long -> NUMERIC(9,6) (~11cm precision); rating -> NUMERIC(2,1); description -> TEXT; opening_time -> TIME; created_at -> TIMESTAMPTZ.

Answer

✅ ANSWER

```
CREATE TABLE public.restaurants (
  restaurant_id SERIAL PRIMARY KEY,
  name          VARCHAR(150),
  city          VARCHAR(50),
  latitude      NUMERIC(9,6),      -- ~11cm precision
  longitude     NUMERIC(9,6),
  avg_rating    NUMERIC(2,1),      -- 4.3 ok, blocks 42.3
  is_pure_veg   BOOLEAN,
  opening_time  TIME,
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE public.menu_items (
  item_id           SERIAL PRIMARY KEY,
  restaurant_id     INT,
  item_name         VARCHAR(120),
  description       TEXT,           -- up to 500 words
  price             NUMERIC(10,2),
  is_veg            BOOLEAN,
  prep_time_minutes INT,
  calories          INT
);
```

CRAZY

SCENARIO: A real migration: a TEXT-everything table already has data loaded. Convert EVERY column to its proper type in place - exactly what teams do when they inherit a badly-designed table.

YOUR TASK:

a) : Create shop.legacy_orders (order_id, order_date, total, is_paid) - all TEXT - and INSERT 3 rows of clean text values.
b) : ALTER order_id TEXT->INT, order_date TEXT->DATE, total TEXT->NUMERIC(10,2), is_paid TEXT->BOOLEAN (each with USING).
c) : Now prove the fix worked: SELECT with WHERE total > 900 (numeric!) and ORDER BY order_date - both behave correctly now.
d) : Bonus: try inserting total = 'abc' AFTER the conversion - the NUMERIC type now rejects it.

Hint: This is the whole arc: yesterday's TEXT pain -> today's ALTER ... USING -> correct math and sorting forever. Keep the loaded data text-clean so every USING cast succeeds.

Answer (1/2)

✅ ANSWER

```
CREATE SCHEMA IF NOT EXISTS shop;
CREATE TABLE shop.legacy_orders (
  order_id   TEXT,
  order_date TEXT,
  total      TEXT,
  is_paid    TEXT
);
INSERT INTO shop.legacy_orders VALUES
  ('1', '2026-01-10', '900',  'true'),
  ('2', '2026-01-05', '1000', 'false'),
  ('3', '2026-01-20', '4999', 'true');

ALTER TABLE shop.legacy_orders
  ALTER COLUMN order_id   TYPE INT           USING order_id::int;
ALTER TABLE shop.legacy_orders
  ALTER COLUMN order_date TYPE DATE          USING order_date::date;
ALTER TABLE shop.legacy_orders
  ALTER COLUMN total      TYPE NUMERIC(10,2) USING total::numeric;
ALTER TABLE shop.legacy_orders
  ALTER COLUMN is_paid    TYPE BOOLEAN       USING is_paid::boolean;

-- Now numbers and dates behave correctly:
```

Answer (2/2)

✅ ANSWER

```
SELECT order_id, total FROM shop.legacy_orders
WHERE total > 900 ORDER BY order_date;
-- '1000' and '4999' both return (real numeric comparison).
```

STORY

The Exam Hall Without Rules

A school runs an exam with NO rules. Chaos:

- Two students sit in the same seat - fight (no UNIQUE)
- A student's name is blank on the admit card (no NOT NULL)
- Roll number -47 is accepted (no CHECK)
- Hall 99 does not exist but the card says it (no FOREIGN KEY)

The vice-principal fixes it: every seat a unique number (PRIMARY KEY), every student a name (NOT NULL), positive rolls only (CHECK), hall numbers must match the real list (FOREIGN KEY). Next exam: zero chaos. Those rules are CONSTRAINTS.

Application code can be bypassed. Database constraints CANNOT. A single CHECK (price > 0) can save crores.

DEFINITION

NOT NULL & DEFAULT

NOT NULL: the column can never be empty - any INSERT/UPDATE setting NULL is rejected. DEFAULT: if you do not provide a value, PostgreSQL auto-fills one. Together they mean 'this column always has a sensible value, even if the developer forgets'.

[Syntax + Worked Example]

```
CREATE TABLE day_4.orders (
  order_id      SERIAL PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  order_status  VARCHAR(20)  NOT NULL DEFAULT 'pending',
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);


-- Provide only the required value; defaults fill the rest:
INSERT INTO day_4.orders (customer_name) VALUES ('Priya');
-- order_status='pending', created_at=now (auto-filled)


-- FAILS: customer_name is NOT NULL and missing
INSERT INTO day_4.orders (order_status) VALUES ('shipped');
-- ERROR: null value in column "customer_name" violates not-null constraint
```

DEFINITION

UNIQUE & CHECK (1/2)

UNIQUE: no two rows may share the value (emails, phones, PAN). Multiple NULLs ARE allowed - NULL != NULL in PostgreSQL. CHECK: a custom rule evaluated on every write; if it is false, the write is rejected.

DEFINITION

UNIQUE & CHECK (2/2)

[Syntax + Worked Example]

```
CREATE TABLE day_4.signups (
  signup_id SERIAL PRIMARY KEY,
  email     VARCHAR(150) UNIQUE NOT NULL,
  age       INT CHECK (age >= 18),
  status    VARCHAR(20) NOT NULL DEFAULT 'active'
    CHECK (status IN ('active','paused','closed')),
  -- table-level CHECK can span columns:
  start_date DATE, end_date DATE,
  CONSTRAINT valid_dates CHECK (end_date >= start_date)
);


INSERT INTO day_4.signups (email, age, start_date, end_date)
VALUES ('priya@gmail.com', 25, '2026-01-01', '2026-12-31');  -- OK


-- FAILS: duplicate email (UNIQUE)
-- FAILS: age 16 (CHECK age >= 18)
-- FAILS: status 'banned' (CHECK IN list)
```

DEFINITION

PRIMARY KEY - The Identity of Every Row

PRIMARY KEY = UNIQUE + NOT NULL. It uniquely identifies each row, only ONE per table. Usually a SERIAL surrogate id (immutable, compact). Use UNIQUE - not PRIMARY KEY - for natural identifiers like email (those change; a PK should never change).

```
[Syntax + Composite Key]
-- Single-column (most common):
CREATE TABLE day_4.customers (
  customer_id SERIAL PRIMARY KEY,
  email       VARCHAR(150) UNIQUE NOT NULL
);


-- Composite PK - the COMBINATION must be unique
-- (one attendance row per employee per day):
CREATE TABLE day_4.attendance (
  emp_id   INT  NOT NULL,
  att_date DATE NOT NULL,
  status   VARCHAR(10) NOT NULL DEFAULT 'present',
  PRIMARY KEY (emp_id, att_date)
);
-- inserting the same (emp_id, att_date) twice -> duplicate key error
```

DEFINITION

FOREIGN KEY - The Bridge Between Tables (1/2)

A FOREIGN KEY column references the PRIMARY KEY of another table. It enforces referential integrity: a child row can only reference a parent that exists - no orphan records. Insert the parent BEFORE the child.

[Syntax + Worked Example]

```
CREATE TABLE day_4.rm_customers (
  customer_id SERIAL PRIMARY KEY,
  full_name   VARCHAR(100) NOT NULL,
  email       VARCHAR(150) UNIQUE NOT NULL
);
```

DEFINITION

FOREIGN KEY - The Bridge Between Tables (2/2)

```
CREATE TABLE day_4.rm_orders (
  order_id    SERIAL PRIMARY KEY,
  customer_id INT NOT NULL
      REFERENCES day_4.rm_customers(customer_id),
  order_total NUMERIC(10,2) NOT NULL CHECK (order_total > 0)
);


INSERT INTO day_4.rm_customers (full_name, email)
VALUES ('Meera', 'meera@gmail.com');
INSERT INTO day_4.rm_orders (customer_id, order_total)
VALUES (1, 2499.00);   -- OK, customer 1 exists


-- FAILS: customer 999 does not exist (orphan blocked)
INSERT INTO day_4.rm_orders (customer_id, order_total) VALUES (999, 500);
```

DEFINITION

ON DELETE Actions - What Happens to Children?

When you delete a parent row that still has children, the FK's ON DELETE clause decides the outcome:

- RESTRICT (default): block the delete until children are handled. Safest - use for financial/critical data.
- CASCADE: delete the children too. For disposable dependents (comments on a deleted post).
- SET NULL: keep the children, set their FK to NULL. For reassignable links (employee's manager leaves).

[Syntax]

```
FOREIGN KEY (post_id) REFERENCES day_4.blog_posts(post_id)
    ON DELETE CASCADE;


manager_id INT REFERENCES day_4.managers(manager_id)
    ON DELETE SET NULL;   -- FK column must allow NULL


dept_id INT NOT NULL REFERENCES day_4.departments(dept_id)
    ON DELETE RESTRICT;   -- the safe default
```

DEFINITION

ALTER TABLE - Add & Drop Constraints on EXISTING Tables (1/2)

You do NOT have to recreate a table to add or remove a rule. Every constraint can be added or dropped later with ALTER TABLE - this is how real databases evolve their rules. (Same tables you altered TYPE on earlier today.)

[NOT NULL and DEFAULT (per column)]

```
-- NOT NULL  (fails if existing rows already hold NULL - clean them first)
ALTER TABLE t ALTER COLUMN c SET NOT NULL;
ALTER TABLE t ALTER COLUMN c DROP NOT NULL;


-- DEFAULT  (affects only FUTURE inserts; existing rows are untouched)
ALTER TABLE t ALTER COLUMN c SET DEFAULT 'pending';
ALTER TABLE t ALTER COLUMN c DROP DEFAULT;
```

DEFINITION

ALTER TABLE - Add & Drop Constraints on EXISTING Tables (2/2)

```
[UNIQUE / CHECK / PRIMARY KEY / FOREIGN KEY (named constraints)]
-- UNIQUE
ALTER TABLE t ADD CONSTRAINT uq_t_email UNIQUE (email);
ALTER TABLE t DROP CONSTRAINT uq_t_email;


-- CHECK  (single- or multi-column)
ALTER TABLE t ADD CONSTRAINT chk_t_price CHECK (price >= 0);
ALTER TABLE t DROP CONSTRAINT chk_t_price;


-- PRIMARY KEY
ALTER TABLE t ADD CONSTRAINT pk_t PRIMARY KEY (id);
ALTER TABLE t DROP CONSTRAINT pk_t;


-- FOREIGN KEY  (with an ON DELETE action)
ALTER TABLE child ADD CONSTRAINT fk_child_parent
  FOREIGN KEY (parent_id) REFERENCES parent(id) ON DELETE CASCADE;
ALTER TABLE child DROP CONSTRAINT fk_child_parent;
```

PRACTICE BLOCK 2 - Constraints & Keys

All practice in accio_NN (schema day_4). The failing INSERTs are intentional - read the error to see which rule fired.

EASY

SCENARIO: A food-delivery app needs every order to have a customer name, with status and timestamp auto-filled even when the app crashes mid-checkout.

YOUR TASK:

a) : Create day_4.orders: order_id SERIAL PK, customer_name NOT NULL, order_status DEFAULT 'pending', created_at DEFAULT now().
b) : INSERT a row with only customer_name; SELECT it and confirm the defaults filled in.
c) : Try an INSERT with no customer_name - observe the NOT NULL error.

Hint: DEFAULT applies only when you OMIT the column. NOT NULL + DEFAULT = always-present value, even if the developer forgets it.

Answer

✅ ANSWER

```
CREATE SCHEMA IF NOT EXISTS day_4;
CREATE TABLE day_4.orders (
  order_id      SERIAL PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  order_status  VARCHAR(20)  NOT NULL DEFAULT 'pending',
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);

INSERT INTO day_4.orders (customer_name) VALUES ('Priya Mehta');
SELECT * FROM day_4.orders;   -- status='pending', created_at filled

-- FAILS: customer_name missing
INSERT INTO day_4.orders (order_status) VALUES ('shipped');
-- ERROR: null value in column "customer_name" violates not-null constraint
```

MEDIUM

SCENARIO: An order-tracking system must only accept known statuses and ratings of 1-5, and never duplicate an order's tracking email.

YOUR TASK:

a) : Create day_4.order_tracking: tracking_id SERIAL PK, contact_email UNIQUE NOT NULL, status CHECK IN ('placed','shipped','delivered','cancelled'), rating CHECK BETWEEN 1 AND 5.
b) : INSERT one valid row.
c) : Show three failing INSERTs: a duplicate email, status 'dispatched', and rating 7.

Hint: Each constraint is checked independently. CHECK (status IN (...)) is a dropdown; CHECK (rating BETWEEN 1 AND 5) is a range.

Answer

✅ ANSWER

```
CREATE TABLE day_4.order_tracking (
  tracking_id   SERIAL PRIMARY KEY,
  contact_email VARCHAR(150) UNIQUE NOT NULL,
  status        VARCHAR(20) NOT NULL
      CHECK (status IN ('placed','shipped','delivered','cancelled')),
  rating        INT CHECK (rating BETWEEN 1 AND 5)
);

INSERT INTO day_4.order_tracking (contact_email, status, rating)
VALUES ('a@shop.com', 'shipped', 4);   -- OK

-- FAILS (UNIQUE): duplicate email
INSERT INTO day_4.order_tracking (contact_email, status) VALUES ('a@shop.com','placed');
-- FAILS (CHECK): 'dispatched' not allowed
INSERT INTO day_4.order_tracking (contact_email, status) VALUES ('b@shop.com','dispatched');
-- FAILS (CHECK): rating 7 out of range
INSERT INTO day_4.order_tracking (contact_email, status, rating) VALUES ('c@shop.com','placed',7);
```

HARD

SCENARIO: RetailMart needs orders that always belong to a real customer - no ghost orders pointing at non-existent people.

YOUR TASK:

a) : Create day_4.rm_customers (customer_id SERIAL PK, full_name NOT NULL, email UNIQUE NOT NULL).
b) : Create day_4.rm_orders with customer_id INT NOT NULL REFERENCES rm_customers, and order_total NUMERIC CHECK > 0.
c) : Insert a customer, then a valid order for them.
d) : Try an order for customer_id = 999 - observe the FK error. Then try a duplicate customer email.

Hint: Parent before child: insert the customer first. The FK type (INT) must match the parent PK (SERIAL = INT).

Answer

✅ ANSWER

```
CREATE TABLE day_4.rm_customers (
  customer_id SERIAL PRIMARY KEY,
  full_name   VARCHAR(100) NOT NULL,
  email       VARCHAR(150) UNIQUE NOT NULL
);
CREATE TABLE day_4.rm_orders (
  order_id    SERIAL PRIMARY KEY,
  customer_id INT NOT NULL REFERENCES day_4.rm_customers(customer_id),
  order_total NUMERIC(10,2) NOT NULL CHECK (order_total > 0)
);

INSERT INTO day_4.rm_customers (full_name, email)
VALUES ('Meera Reddy', 'meera@gmail.com');
INSERT INTO day_4.rm_orders (customer_id, order_total) VALUES (1, 2499.00); -- OK

-- FAILS (FK): customer 999 does not exist
INSERT INTO day_4.rm_orders (customer_id, order_total) VALUES (999, 500.00);
-- FAILS (UNIQUE): duplicate email
INSERT INTO day_4.rm_customers (full_name, email) VALUES ('X','meera@gmail.com');
```

CRAZY

SCENARIO: A blogging platform: deleting a post should auto-remove its comments (they are meaningless alone), but deleting a department must be BLOCKED while employees still belong to it. Pick the right ON DELETE policy for each.

YOUR TASK:

a) : Create day_4.blog_posts + day_4.post_comments with ON DELETE CASCADE. Insert a post with 3 comments, delete the post, prove comments are gone (count = 0).
b) : Create day_4.departments + day_4.dept_employees with ON DELETE RESTRICT. Insert a department + an employee, then try to delete the department - observe it is blocked.
c) : Explain in one line why CASCADE fits comments but RESTRICT fits departments (and why financial records should use RESTRICT).

Hint: Ask: if the parent disappears, does the child still make sense alone? No -> CASCADE. Yes / must be preserved -> RESTRICT.

Answer (1/2)

✅ ANSWER

```
-- (a) CASCADE: comments die with the post
CREATE TABLE day_4.blog_posts (
  post_id SERIAL PRIMARY KEY, title VARCHAR(200) NOT NULL);
CREATE TABLE day_4.post_comments (
  comment_id SERIAL PRIMARY KEY,
  post_id INT NOT NULL REFERENCES day_4.blog_posts(post_id) ON DELETE CASCADE,
  comment_text TEXT NOT NULL);
INSERT INTO day_4.blog_posts (title) VALUES ('Learn SQL');
INSERT INTO day_4.post_comments (post_id, comment_text)
VALUES (1,'Great!'),(1,'Helpful!'),(1,'Bookmarked.');
DELETE FROM day_4.blog_posts WHERE post_id = 1;
SELECT count(*) FROM day_4.post_comments;  -- 0 (auto-deleted)

-- (b) RESTRICT: cannot delete a department with employees
CREATE TABLE day_4.departments (
  dept_id SERIAL PRIMARY KEY, dept_name VARCHAR(50) UNIQUE NOT NULL);
CREATE TABLE day_4.dept_employees (
  emp_id SERIAL PRIMARY KEY, emp_name VARCHAR(100) NOT NULL,
  dept_id INT NOT NULL REFERENCES day_4.departments(dept_id) ON DELETE RESTRICT);
INSERT INTO day_4.departments (dept_name) VALUES ('Engineering');
INSERT INTO day_4.dept_employees (emp_name, dept_id) VALUES ('Arun', 1);
-- FAILS (RESTRICT): employees still reference dept 1
```

Answer (2/2)

✅ ANSWER

```
DELETE FROM day_4.departments WHERE dept_id = 1;
```

RECAP

Key Takeaways (1/2)

1. The right type is defense: NUMERIC for money (never FLOAT), VARCHAR for phone/pin (never INT), DATE/TIMESTAMP for time, BOOLEAN for yes/no, SERIAL for auto-ids.

2. ALTER … TYPE … USING: Convert yesterday's TEXT columns to proper types in place. A failing cast forces you to clean bad data - the type defending itself.

3. NOT NULL & DEFAULT: NOT NULL forbids empty; DEFAULT auto-fills. Together = always a sensible value.

RECAP

Key Takeaways (2/2)

4. UNIQUE & CHECK: UNIQUE blocks duplicates (multiple NULLs allowed). CHECK enforces custom rules; pair with NOT NULL since NULL passes a CHECK.

5. PRIMARY KEY: UNIQUE + NOT NULL, one per table. Prefer a SERIAL surrogate; use UNIQUE for natural ids like email.

6. FOREIGN KEY: Child references an existing parent - no orphans. Parent first, child second. FK type must match parent PK type.

7. ON DELETE: RESTRICT (block, safest), CASCADE (delete children), SET NULL (unlink). Financial data -> RESTRICT + soft-delete.

Data Types, Constraints & Keys - Quick Reference

-- Core Data Types

```
SERIAL        auto-id (1,2,3...)
INT / BIGINT  whole numbers / math
NUMERIC(p,s)  money / exact decimals
VARCHAR(n)    short text; TEXT = long text
DATE          calendar day
TIMESTAMP(TZ) exact moment
BOOLEAN       true / false
```

-- ALTER TEXT -> Proper Type

```
ALTER TABLE t ALTER COLUMN c
  TYPE INT USING c::int;
ALTER TABLE t ALTER COLUMN c
  TYPE DATE USING c::date;
ALTER TABLE t ALTER COLUMN c
  TYPE NUMERIC(10,2) USING c::numeric;
```

-- Column Constraints

```
col TYPE NOT NULL
col TYPE DEFAULT value
col TYPE UNIQUE
col TYPE CHECK (col > 0)
CHECK (status IN ('a','b'))
CONSTRAINT name CHECK (end >= start)
```

Data Types, Constraints & Keys - Quick Reference

-- Keys

```
id SERIAL PRIMARY KEY
PRIMARY KEY (col_a, col_b)  -- composite
fk INT REFERENCES parent(id)
FOREIGN KEY (fk) REFERENCES parent(id)
email VARCHAR(150) UNIQUE  -- natural id
```

-- ON DELETE Actions

```
... REFERENCES p(id) ON DELETE RESTRICT
... REFERENCES p(id) ON DELETE CASCADE
... REFERENCES p(id) ON DELETE SET NULL
RESTRICT = block (safe default)
CASCADE  = delete children
SET NULL = keep children, unlink
```

-- Golden Rules

```
Money -> NUMERIC, never FLOAT
Phone/pin -> VARCHAR, never INT
Every table needs a PRIMARY KEY
FK type must match parent PK type
Pair CHECK with NOT NULL
Financial FKs -> RESTRICT + soft-delete
```

INTERVIEW QUESTION BANK - Data Types, Constraints & Keys

Practice answering OUT LOUD. Interviews test how clearly you explain.

INTERVIEW

Interview Questions (1/4)

Q1: 'Why never store money as FLOAT?'

A: 'FLOAT/REAL are binary approximations - 0.1+0.2 becomes 0.30000001. Over millions of transactions the drift becomes real losses. NUMERIC stores exact decimals, so money math is always correct.'

Q2: 'Why store phone numbers as VARCHAR, not INT?'

A: 'You never do math on a phone number. It can contain +, dashes, and leading zeros that INT would strip - 098765 becomes 98765. Any identifier you never compute on should be text.'

INTERVIEW

Interview Questions (2/4)

Q3: 'How do you change a TEXT column to NUMERIC when data exists?'

A: 'ALTER TABLE t ALTER COLUMN c TYPE NUMERIC USING c::numeric. The USING clause casts existing values. If a value cannot cast, the ALTER fails - which forces me to clean the bad data first.'

Q4: 'Difference between UNIQUE and PRIMARY KEY?'

A: 'A table can have many UNIQUE constraints but only one PRIMARY KEY. UNIQUE allows NULLs; PRIMARY KEY is UNIQUE + NOT NULL. PK is the row's definitive identity; UNIQUE is business-level uniqueness like email.'

INTERVIEW

Interview Questions (3/4)

Q5: 'Does CHECK (rating BETWEEN 1 AND 5) block NULL?'

A: 'No. NULL makes the expression UNKNOWN, and CHECK only rejects FALSE. To block NULL too, add NOT NULL.'

Q6: 'What is an orphaned record and how do you prevent it?'

A: 'A child row whose FK points to a parent that does not exist - e.g. an order for customer 999 when no customer 999 exists. A FOREIGN KEY constraint prevents it: the database rejects any insert that would create an orphan.'

Q7: 'Natural key or surrogate key?'

A: 'Almost always surrogate (SERIAL/UUID). Natural keys like email change, are large, and leak business data. Surrogates are immutable, compact, and meaningless - ideal PKs. Keep the natural id as UNIQUE.'

INTERVIEW

Interview Questions (4/4)

Q8: 'When CASCADE vs RESTRICT on a foreign key?'

A: 'CASCADE for dependents with no standalone meaning - comments on a deleted post. RESTRICT for data that must be preserved - orders of a closing customer. In fintech, RESTRICT plus soft-delete is the safe default.'

REVEAL

Data Types, Constraints & Keys Self-Check (12 Questions)

Close your notes. Answer from memory:

- 1. Which type for money, and which must you never use?
- 2. Why is phone VARCHAR, not INT? Give 2 reasons.
- 3. Write the ALTER that converts a TEXT column to DATE.
- 4. What does the USING clause do in ALTER … TYPE?
- 5. Difference between NOT NULL and DEFAULT?
- 6. Does a CHECK constraint block NULL? How do you also block NULL?
- 7. UNIQUE vs PRIMARY KEY - two differences.
- 8. What is PRIMARY KEY equal to (in terms of other constraints)?
- 9. What does a FOREIGN KEY guarantee?
- 10. Parent or child first when inserting? Why?
- 11. RESTRICT vs CASCADE vs SET NULL - one use case each.
- 12. Why use RESTRICT + soft-delete for financial data?

Score 10+ -> ready for Filtering & Sorting. Below 10 -> re-read the weak sections.

CHALLENGE

Data Types, Constraints & Keys Take-Home Challenge

Complete these BEFORE Filtering & Sorting, in accio_NN:

1. Convert: : Take the TEXT tables you built on Day 3 and ALTER each column to its proper type (USING casts). Confirm with information_schema.columns that none are 'text' anymore.
2. Constrain: : Add NOT NULL, DEFAULT, UNIQUE, and at least one CHECK to your products/customers/orders tables.
3. Keys: : Give every table a PRIMARY KEY. Add a FOREIGN KEY from orders to customers.
4. Prove it: : Write 4 INSERTs that each fail for a DIFFERENT reason (NOT NULL, UNIQUE, CHECK, FK). Save the error messages.
5. Cascade: : Add an order_items table with ON DELETE CASCADE to orders. Delete an order; confirm its items vanish.

Tomorrow: Filtering & Sorting - your FIRST real DQL. WHERE, LIKE, BETWEEN, IN, IS NULL, ORDER BY, LIMIT on RetailMart.

YOUR SQL JOURNEY

- ✓ SQL Intro
- ✓ Installation
- ✓ DDL + DML
- 4 **Data Types + Constraints <- YOU ARE HERE** ~~<-~~ HERE
- 5 Filtering (first DQL!)
- 9 Joins
- 13 CTEs
- 16 Window Functions
- 20 Indexing
- 25 Views
- 27 Cohort/RFM
- 29 Capstone

STORY

Coming Tomorrow

You have built tables, given them proper types, and protected them with constraints and keys. But how do you FIND a specific customer? List orders above Rs. 5,000? See the top 10 products by price?

Filtering & Sorting introduces WHERE, LIKE, BETWEEN, IN, IS NULL, ORDER BY, and LIMIT - run against the REAL RetailMart data (accio_retailmart_NN). This is where SQL starts to feel POWERFUL - and where you stop building and start ASKING QUESTIONS.

- See you on Filtering & Sorting! Siraj Sir